

Day 6 - Sort LinkedList

ITSRUNTYM

Sort a Linked List

Given the head of a linked list, return it sorted in ascending order.

Possible Approaches

Approach	Idea	Time Complexity	Space Complexity	Notes
Brute Force (Array)	Copy all values to array, sort using Arrays.sort, rebuild list	$O(n \log n)$	$O(n)$	Easy but uses extra space
Merge Sort (Linked List)	Recursively split into halves, merge sorted halves	$O(n \log n)$	$O(\log n)$ recursion stack	Most optimal for singly linked list
Insertion Sort	Insert each node in sorted position in new list	$O(n^2)$	$O(1)$	Good for almost sorted lists

Brute Force Approach

Idea:

1. Traverse list, store values in an array.
2. Sort the array.
3. Rebuild the list from sorted values.

Code:

```
class Solution {
    public ListNode sortList(ListNode head) {
        if (head == null) return null;
        // Step 1: Store values in array
        List<Integer> values = new ArrayList<>();
```

```

        ListNode curr = head;
        while (curr != null) {
            values.add(curr.val);
            curr = curr.next;
        }
// Step 2: Sort the array
        Collections.sort(values);
// Step 3: Rebuild linked list
        curr = head;
        for (int val : values) {
            curr.val = val;
            curr = curr.next;
        }
return head;
    }
}

```

Complexity:

- Time: $O(n \log n)$
- Space: $O(n)$

Merge Sort (Optimized)

Why merge sort works best for linked lists:

- Linked lists have $O(1)$ insertion, but finding middle is $O(n)$, so quicksort is less efficient.
- Merge sort only needs **pointer rearrangement**, no extra array.

Steps:

1. Find middle using **slow/fast pointer**.
2. Recursively sort left and right halves.
3. Merge sorted halves.

Code:

```

class Solution {
    public ListNode sortList(ListNode head) {
        if (head == null || head.next == null) return head;

// Step 1: Split list into two halves
        ListNode mid = getMid(head);
        ListNode right = mid.next;
        mid.next = null;

// Step 2: Sort each half
        ListNode leftSorted = sortList(head);

```

```

        ListNode rightSorted = sortList(right);
// Step 3: Merge
        return merge(leftSorted, rightSorted);
    }

private ListNode getMid(ListNode head) {
    ListNode slow = head, fast = head.next;
    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
    }
    return slow;
}

private ListNode merge(ListNode l1, ListNode l2) {
    ListNode dummy = new ListNode(0);
    ListNode curr = dummy;
    while (l1 != null && l2 != null) {
        if (l1.val < l2.val) {
            curr.next = l1;
            l1 = l1.next;
        } else {
            curr.next = l2;
            l2 = l2.next;
        }
        curr = curr.next;
    }
    curr.next = (l1 != null) ? l1 : l2;
    return dummy.next;
}
}

```

Complexity:

- Time: $O(n \log n)$
- Space: $O(\log n)$ (recursion stack)

Insertion Sort

When to use: When the list is **nearly sorted**, giving $O(n)$ in best case.

Steps:

1. Create a dummy head for the sorted part.
2. Iterate nodes, inserting each into the correct sorted position.

Code:

```

class Solution {
    public ListNode insertionSortList(ListNode head) {
        ListNode dummy = new ListNode(0);
        ListNode curr = head;
        while (curr != null) {
            ListNode prev = dummy;
            // Find correct position
            while (prev.next != null && prev.next.val < curr.val) {
                prev = prev.next;
            }
            ListNode nextTemp = curr.next;
            curr.next = prev.next;
            prev.next = curr;
            curr = nextTemp;
        }
        return dummy.next;
    }
}

```

Complexity:

- Time: $O(n^2)$ worst case
- Space: $O(1)$

Example Walkthrough – Merge Sort

Input: $4 \rightarrow 2 \rightarrow 1 \rightarrow 3$

1. Middle = 2 → Split: $4 \rightarrow 2$ and $1 \rightarrow 3$
2. Sort each half:
 - $4 \rightarrow 2 \rightarrow 2 \rightarrow 4$
 - $1 \rightarrow 3 \rightarrow 1 \rightarrow 3$
3. Merge: $1 \rightarrow 2 \rightarrow 3 \rightarrow 4$

Related LeetCode Questions

1. 148. Sort List – (Merge sort linked list) <https://leetcode.com/problems/sort-list/description/>.
1. 147. Insertion Sort List – (Insertion sort) <https://leetcode.com/problems/insertion-sort-list/description/>.